Check for updates

ACM DIGITAL LIBRARY

Association for Computing Machinery

acm open

Latest updates: https://dl.acm.org/doi/10.1145/1094855.1094900

ARTICLE

Predictability by construction

PAULO MERSON, Software Engineering Institute, Pittsburgh, PA, United States

SCOTT HISSAM, Software Engineering Institute, Pittsburgh, PA, United States

Open Access Support provided by:

Software Engineering Institute

PDF Download
1094855.1094900.pdf
11 March 2026
Total Citations: 6
Total Downloads: 408

Published: 16 October 2005

Citation in BibTeX format

OOPSLA05: ACM SIGPLAN Object Oriented Programming Systems and Applications Conference
October 16 - 20, 2005
CA, San Diego, USA

Conference Sponsors:
SIGPLAN

OOPSLA '05: Companion to the 20th annual ACM SIGPLAN conference on Object-oriented programming, systems, languages, and applications (October 2005)
https://doi.org/10.1145/1094855.1094900
ISBN: 1595931937

Predictability by Construction

Paulo Merson
Software Engineering Institute
Pittsburgh, PA 15213-3890, USA
pfm@sei.cmu.edu

Scott Hissam
Software Engineering Institute
Pittsburgh, PA 15213-3890, USA
shissam@sei.cmu.edu

ABSTRACT
The Predictable Assembly from Certifiable Components (PACC) team at the Software Engineering Institute has developed a vehicle to create component-based systems that have predictable behavior prior to implementation. The project has been used to successfully predict performance and safety properties of real-time systems, but the concepts and technology can be applied to predict other properties (e.g., reliability, security).

Categories and Subject Descriptors
D.2.2 [Software Engineering]: Design Tools and Techniques – *Computer-aided software engineering.*

General Terms
Design, Performance, Theory, Verification.

Keywords
Predictable assembly, components, performance, model checking.

1. INTRODUCTION
Many software systems have stringent quality attribute requirements. For example, avionics software has to have high reliability and continue to operate upon a failure, industrial robots have to perform tasks with strict deadlines, magnetic resonance imaging machines have to operate within safety thresholds for radiation. The practice to date has been to verify these requirements through testing—the software system is designed, implemented and then tested to exhaustion. The major issue is the high cost to fix a problem that is found in tests, especially when the problem requires redesigning the system to meet performance, reliability or some other quality attribute requirement.

Our work proposes a different approach: to evaluate the ability of a system to meet quality attribute requirements prior to construction. In other words, predict the behavior of a system looking solely at its design. The PACC team (www.sei.cmu.edu/pacc) at Carnegie Mellon Software Engineering Institute is developing the tools and technology to do that for component-based systems. PACC stands for Predictable Assemblies from Certifiable Components. Behind the name is a simple and powerful idea: predict the behavior of assemblies of components before implementation, based on known, certifiable, properties of the components.

Copyright is held by the author/owner(s).
OOPSLA'05, October 16–20, 2005, San Diego, California, USA.
ACM 1-59593-193-7/05/0010.

2. PREDICTION-ENABLED
We use analytic theories to predict the behavior of component-based systems. Each analytic theory imposes constraints that delimit the class of systems whose behavior can be predicted. If an assembly (a system assembled from components) satisfies the analytic constraints, it is said to be *in the zone* of predictable assemblies. For a predictable assembly, we can create a *model representation* that is an abstraction of the assembly capturing the information relevant to the analysis to be performed (Figure 1).

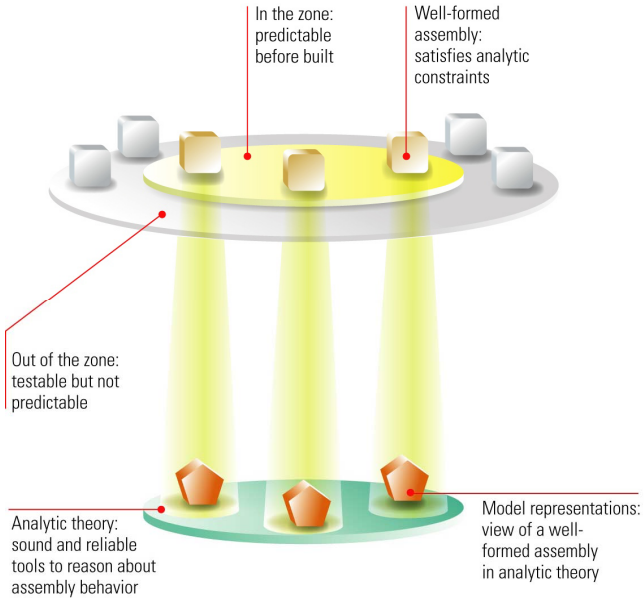

Figure 1. Assemblies in the zone are predictable.

2.1 Reasoning Frameworks
From the user's perspective, all the elements that are necessary to predict properties using a given analytic theory are encapsulated in a *reasoning framework* [1]. The implementation of a reasoning framework takes as input an architecture description of an assembly expressed in an architectural language (e.g. UML). Then *interpretation* transforms the architecture description into a model representation. The next step is to submit the model to an *evaluation procedure*, which is a computable algorithm created based on the analytic theory. The result of the evaluation procedure is the predicted property for that assembly (Figure 2). The key contribution is that a software engineer, who is not an expert in that analytic theory, can use a reasoning framework to reason about properties of the design.

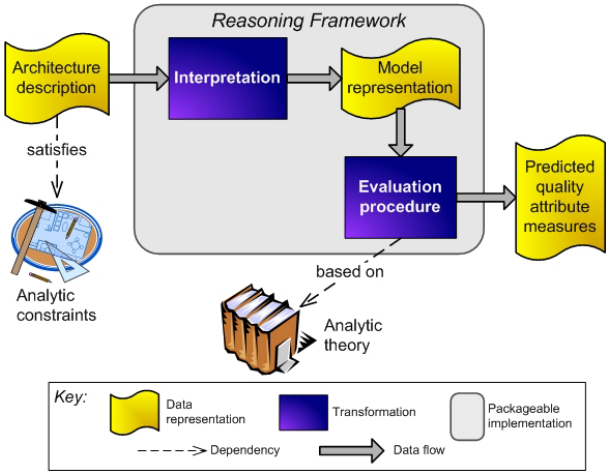

Figure 2. Reasoning framework implementation.

We have developed performance reasoning frameworks to analyze timing properties of real-time systems and predict average latency of tasks (Section 2.2). We also have a reasoning framework based primarily on model checking to reason about safety and liveness properties (Section 2.3). We are currently investigating the creation of security and reliability reasoning frameworks.

2.2 Performance Reasoning Framework

λ_* (Lambda-star) is a suite of performance reasoning frameworks for predicting the average-case latency of periodic and stochastic tasks in what is typified as embedded, real-time control systems. Developed first, the λ_{ABA} reasoning framework focuses on control systems in which all tasks have hard periodic deadlines and are schedulable using Generalized Rate Monotonic Analysis. Now under development, the λ_{SS} reasoning framework (founded on sporadic server scheduling algorithm and real-time queueing theory [2]) extends the first to support control systems in which some tasks have hard periodic deadlines and other tasks have softer deadlines with stochastic (non-periodic) event interarrivals.

2.3 Model-Checking Reasoning Framework

ComFoRT is a reasoning framework that can determine whether systems satisfy behavioral assertions that can be used to characterize safety, reliability, or security requirements [3]. Examples of assertions, which in practice are expressed as temporal logic, include: an industrial robot never enters a work area when sensors indicate a person is in the work area, or message acknowledgements are never lost or sent to the wrong receiver. The analytic theory ComFoRT draws from is temporal logic model checking.

3. TOOL INFRASTRUCTURE

Current work is centered in the evolution and validation[1] of the reasoning frameworks. In parallel, we have developed a tool infrastructure that comprises several elements:

[1] Validation is the process where we compare predicted results to actual behavior of implemented systems to establish objective confidence in the reasoning framework results.

- ***CCL parser and C code generator***: CCL [4] is the language currently used to create the architecture descriptions. We will also support UML 2.0 in a near future.
- ***Performance reasoning framework plug-in***: encapsulates interpretation and evaluation procedure using the Extend simulator and a purely analytic predictor as an Eclipse plug-in.
- ***ComFoRT plug-in***: encapsulates interpretation and evaluation procedure, including the Copper model checker, as an Eclipse plug-in.
- ***PECT IDE***: is an Eclipse-based tool that binds together the CCL editor, parser and code generator, as well as the reasoning framework plug-ins (Figure 3).

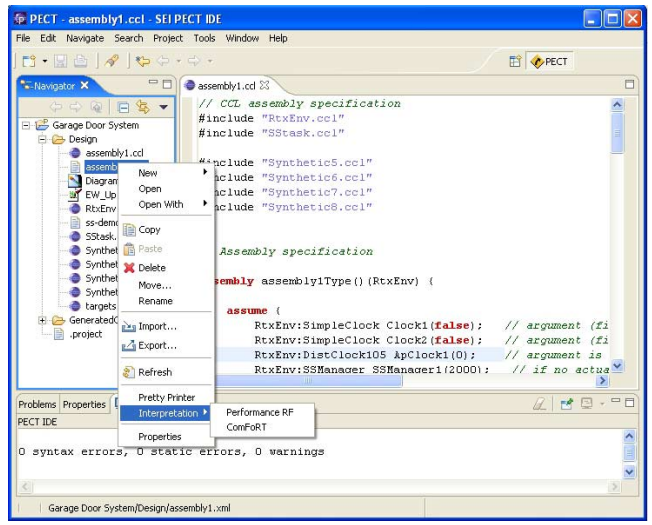

Figure 3. Screenshot of the Eclipse-based IDE.

- ***Pin component technology***: is the basic fabrication mechanism for components and their assemblies.
- ***Measurement infrastructure***: supports collecting timed traces of component and assembly execution in a controlled environment.

4. REFERENCES

[1] Bass, L., Ivers, J., Klein, M, and Merson, P. *Reasoning Frameworks*. CMU/SEI-2005-TR-007, Software Engineering Institute, Pittsburgh, PA, 2005.

[2] Hissam, S., Klein, M., Lehoczky, J., Merson, P., Moreno, G., and Wallnau, K. Performance Property Theories for Predictable Assembly from Certifiable Components (PACC). CMU/SEI-2004-TR-017, Software Engineering Institute, Pittsburgh, PA, 2004.

[3] Ivers, J., and Sharygina, N. *Overview of ComFoRT: A Model Checking Reasoning Framework*. CMU/SEI-2004-TN-018, Software Engineering Institute, Pittsburgh, PA, 2004.

[4] Wallnau, K, and Ivers, J. *Snapshot of CCL: A Language for Predictable Assembly*. CMU/SEI-2003-TN-025, Software Engineering Institute, Pittsburgh, PA, 2003.